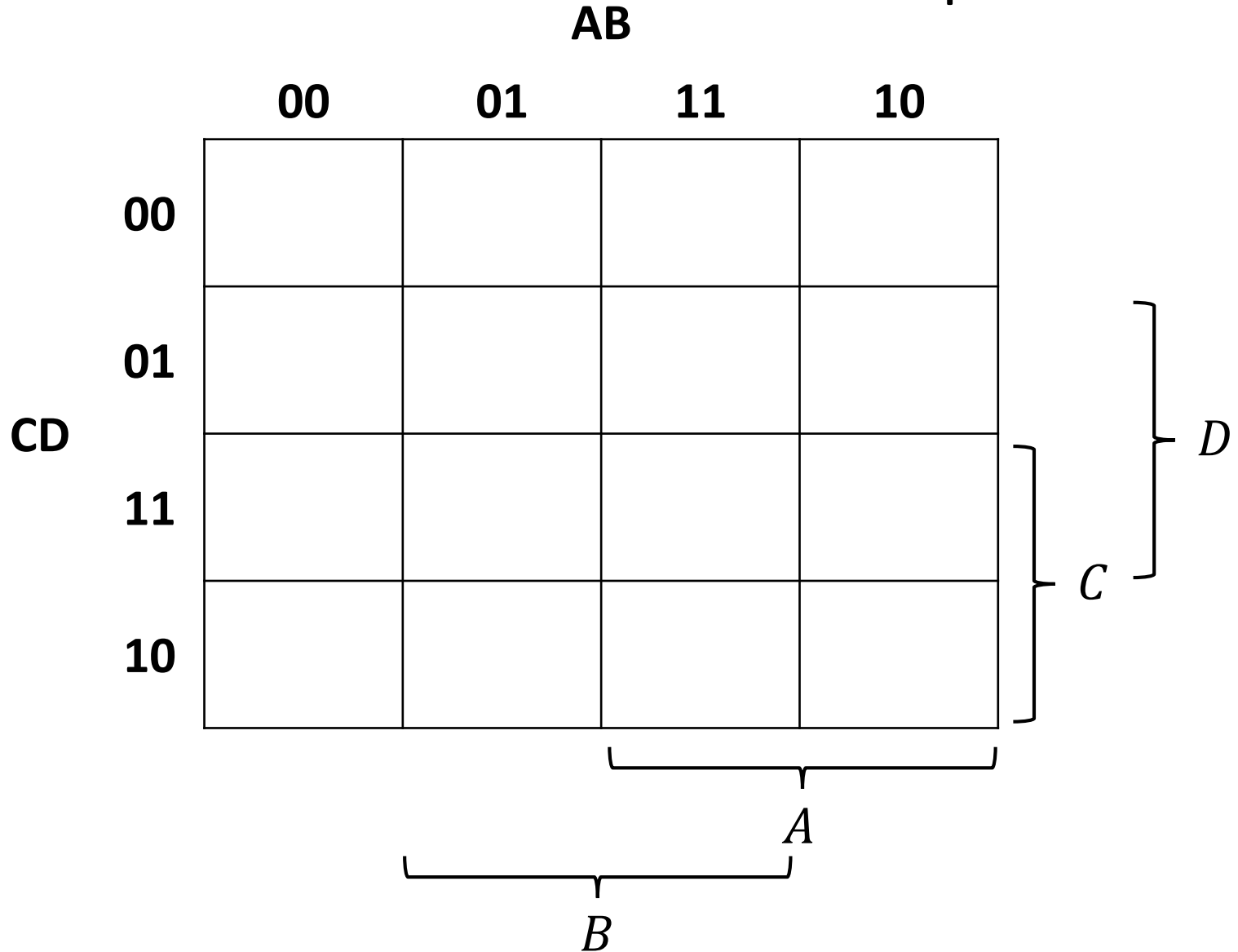


Lecture 5: Logic Design Fundamentals – Part 2

CMPS 221 – Computer Organization and Design

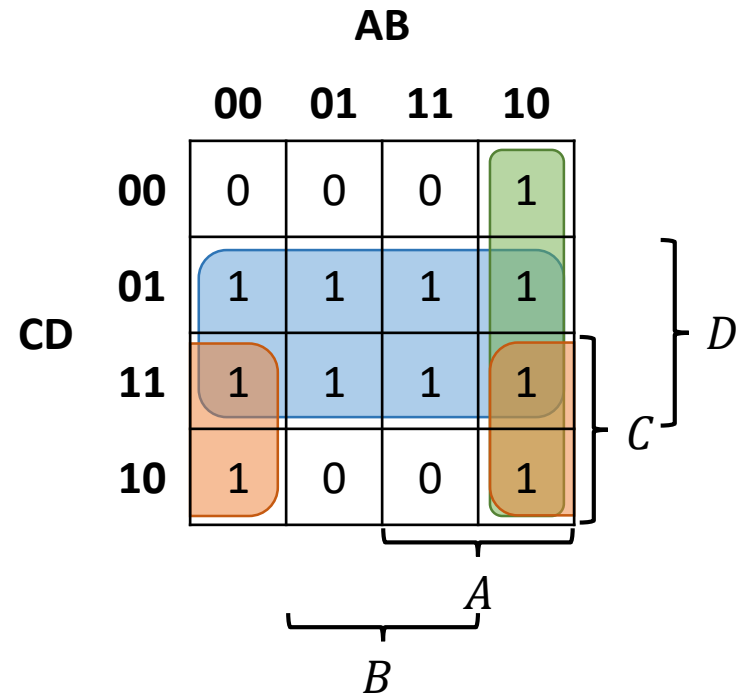
Four Variable K-map



Four Variable K-map Example

Inputs				Output
A	B	C	D	E
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Truth Table



$$E = D + A \cdot \bar{B} + C \cdot \bar{B}$$

K-map

Don't Cares

- A logic block sometimes has input combinations that are known never to occur, hence the corresponding output will never be used
- In this case, the output in the K-map is labelled as “d” for **don't care** and can be treated as 0 or 1 during simplification, whichever is more convenient

Don't Care Example

Inputs				Output
A	B	C	D	E
0	0	0	0	1
0	0	0	1	1
0	0	1	0	
0	0	1	1	1
0	1	0	0	
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	
1	0	1	0	0
1	0	1	1	1
1	1	0	0	
1	1	0	1	1
1	1	1	0	0
1	1	1	1	

Truth Table

		AB			
		00	01	11	10
CD	00	1	d	d	0
	01	1	0	1	d
	11	1	1	d	1
	10	d	0	0	0

$\underbrace{\hspace{10em}}_B$
 $\underbrace{\hspace{10em}}_A$

$\left. \begin{array}{l} \text{Group 1 (Green): } (00,00), (00,01), (11,00), (11,01) \\ \text{Group 2 (Blue): } (01,00), (01,01), (11,00), (11,01) \\ \text{Group 3 (Orange): } (01,11), (01,10), (11,11), (11,10) \\ \text{Group 4 (Brown): } (11,11), (11,10), (10,11), (10,10) \end{array} \right\} \begin{array}{l} D \\ C \end{array}$

$$E = C \cdot D + \bar{A} \cdot \bar{B} + A \cdot D$$

K-map

Canonical Forms

- So far, we have been expressing logic equations in a canonical form, particularly as a **sum of products**
 - Any logic function can be expressed in this form
- Another canonical form is **product of sums**
 - Any logic function can also be expressed in this form

Product of Sums from Truth Table

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

Sum of Products

Inputs			Output
D	K	L	$\bar{B}$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Invert truth table and get sum of products for $\bar{B}$

$$\bar{B} = \bar{D} \cdot \bar{K} \cdot \bar{L} + \bar{D} \cdot K \cdot \bar{L} + \bar{D} \cdot K \cdot L + D \cdot \bar{K} \cdot \bar{L}$$

Apply DeMorgan's

$$B = (D + K + L) \cdot (D + \bar{K} + L) \cdot (D + \bar{K} + \bar{L}) \cdot (\bar{D} + K + L)$$

Product of Sums

Product of Sums from Truth Table

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \underline{\bar{D} \cdot \bar{K} \cdot L} + \underline{D \cdot \bar{K} \cdot L} + \underline{D \cdot K \cdot \bar{L}} + \underline{D \cdot K \cdot L}$$

minterms

Sum of Products

Inputs			Output
D	K	L	$\bar{B}$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Invert truth table and get sum of products for $\bar{B}$

$$\bar{B} = \bar{D} \cdot \bar{K} \cdot \bar{L} + \bar{D} \cdot K \cdot \bar{L} + \bar{D} \cdot K \cdot L + D \cdot \bar{K} \cdot \bar{L}$$

Apply DeMorgan's

$$B = \underline{(D + K + L)} \cdot \underline{(D + \bar{K} + L)} \cdot \underline{(D + \bar{K} + \bar{L})} \cdot \underline{(\bar{D} + K + L)}$$

Product of Sums

maxterms

Minterms and Maxterms

A	B	C	Minterm	Maxterm
0	0	0	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	$A + B + C$
0	0	1	$\bar{A} \cdot \bar{B} \cdot C$	$A + B + \bar{C}$
0	1	0	$\bar{A} \cdot B \cdot \bar{C}$	$A + \bar{B} + C$
0	1	1	$\bar{A} \cdot B \cdot C$	$A + \bar{B} + \bar{C}$
1	0	0	$A \cdot \bar{B} \cdot \bar{C}$	$\bar{A} + B + C$
1	0	1	$A \cdot \bar{B} \cdot C$	$\bar{A} + B + \bar{C}$
1	1	0	$A \cdot B \cdot \bar{C}$	$\bar{A} + \bar{B} + C$
1	1	1	$A \cdot B \cdot C$	$\bar{A} + \bar{B} + \bar{C}$

Sum of Products: sum of minterms for all rows with output 1

Product of Sums: product of maxterms for all rows with output 0

Product of Sums from K-maps

		DK			
		00	01	11	10
L	0	0	0	1	0
	1	1	0	1	1

$\underbrace{\hspace{1.5cm}}_K \underbrace{\hspace{1.5cm}}_D \underbrace{\hspace{1.5cm}}_L$

$$B = \bar{K} \cdot L + D \cdot K$$

Sum of Products

		DK			
		00	01	11	10
L	0	1	1	0	1
	1	0	1	0	0

$\underbrace{\hspace{1.5cm}}_K \underbrace{\hspace{1.5cm}}_D \underbrace{\hspace{1.5cm}}_L$

Invert K-map and get sum of products for $\bar{B}$

$$\bar{B} = \bar{K} \cdot \bar{L} + \bar{D} \cdot K$$

$$B = (K + L) \cdot (D + \bar{K})$$

Apply DeMorgan's Law

Product of Sums

Product of Sums from K-maps

		AB			
		00	01	11	10
CD	00	1	d	d	0
	01	1	0	1	d
	11	1	1	d	1
	10	d	0	0	0

$\underbrace{\hspace{10em}}_B$
 $\underbrace{\hspace{10em}}_C$
 $\underbrace{\hspace{10em}}_D$

$$E = C \cdot D + \bar{A} \cdot \bar{B} + A \cdot D$$

Sum of Products

		AB			
		00	01	11	10
CD	00	0	d	d	1
	01	0	1	0	d
	11	0	0	d	0
	10	d	1	1	1

$\underbrace{\hspace{10em}}_B$
 $\underbrace{\hspace{10em}}_C$
 $\underbrace{\hspace{10em}}_D$

Invert K-map
and get sum of
products for $\bar{E}$
(d's stay the
same)

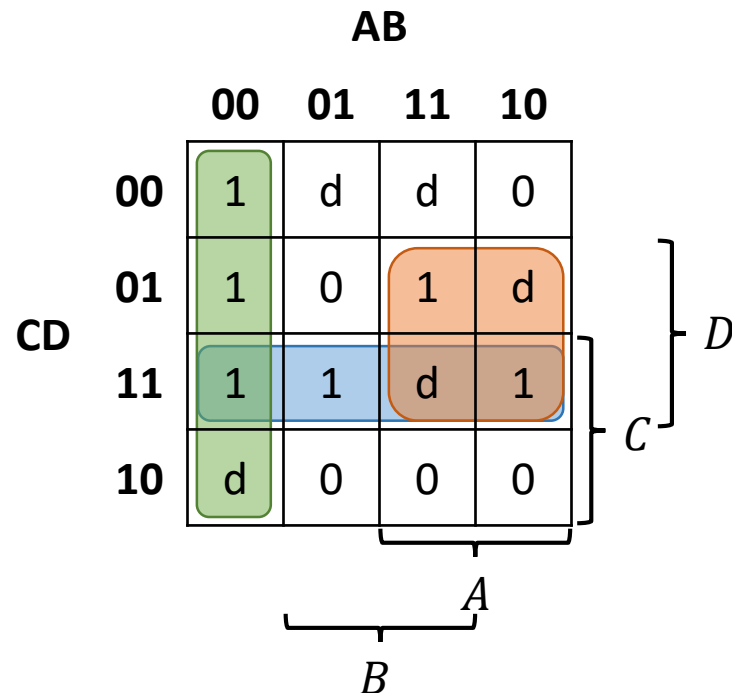
$$\bar{E} = C \cdot \bar{D} + \bar{A} \cdot B \cdot \bar{C} + A \cdot \bar{D}$$

$$E = (\bar{C} + D) \cdot (A + \bar{B} + C) \cdot (\bar{A} + D)$$

Product of Sums

Apply
DeMorgan's
Law

Product of Sums from K-maps

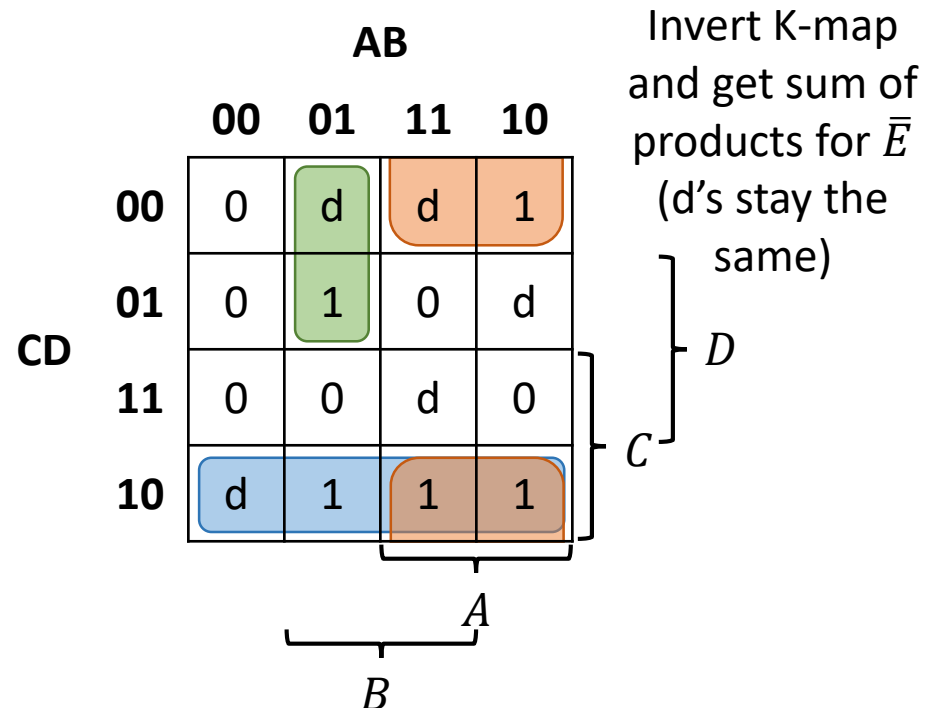


$$E = C \cdot D + \bar{A} \cdot \bar{B} + A \cdot D$$

11 gate inputs in total

Sum of Products

*The sum of products form in this case is more efficient.
In other cases, product of sums might be more efficient.*



Invert K-map
and get sum of
products for $\bar{E}$
(d's stay the
same)

$$\bar{E} = C \cdot \bar{D} + \bar{A} \cdot B \cdot \bar{C} + A \cdot \bar{D}$$

$$E = (\bar{C} + D) \cdot (A + \bar{B} + C) \cdot (\bar{A} + D)$$

13 gate inputs in total

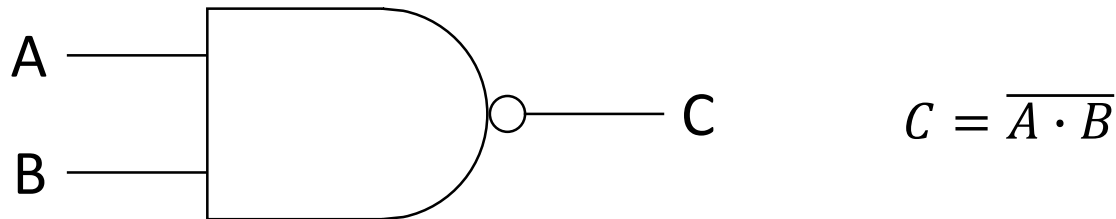
Product of Sums

Universal Gates

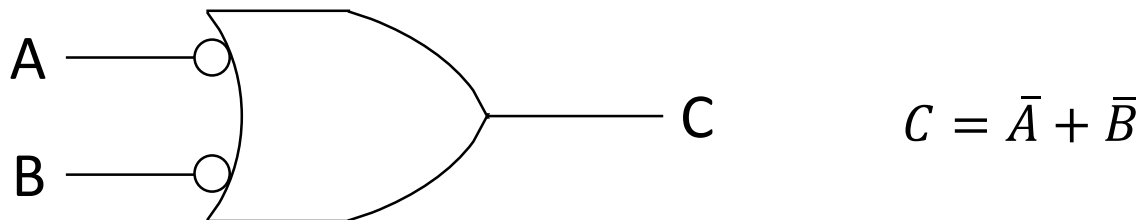
- A **universal gate** is a type of gate that can be used to construct all logic functions
- AND, OR, and NOT gates are not universal gates
- Two common universal gates are NAND and NOR

NAND Gates

- A **NAND Gate** is a logic gate that implements an AND operation followed by a NOT operation
 - Implementing NAND with transistors is cheaper than implementing AND and NOT separately then cascading

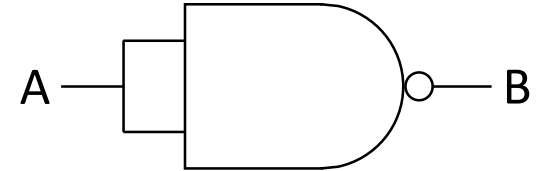
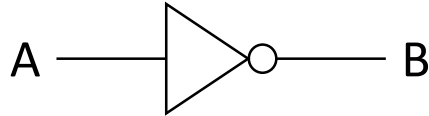


- Applying DeMorgan's laws, a NAND gate is equivalent to an OR gate with inverted inputs

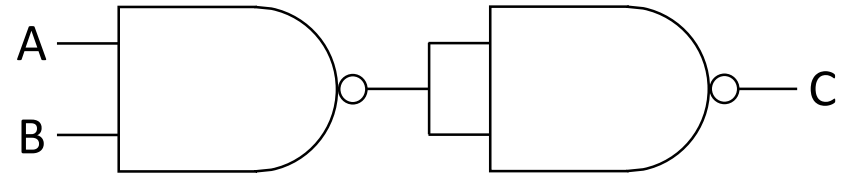
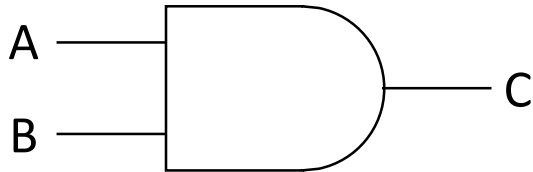


Mapping to NAND Gates

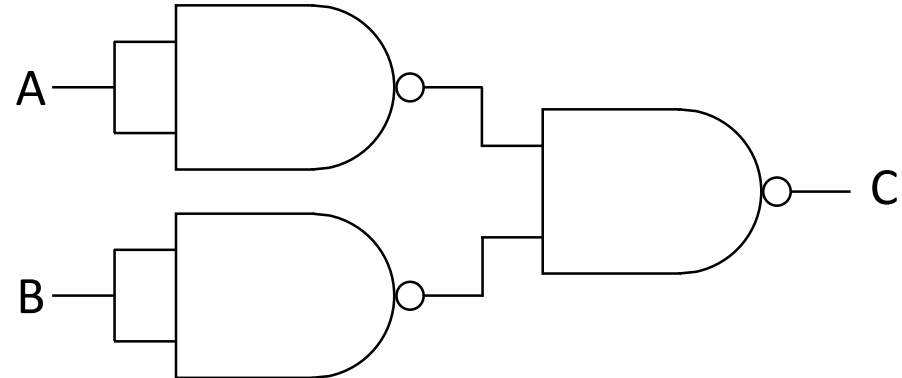
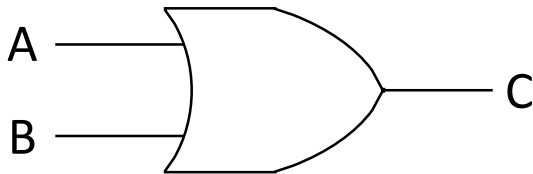
NOT



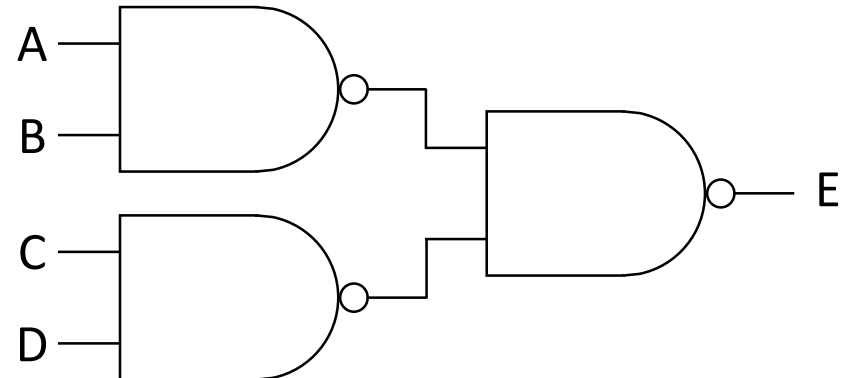
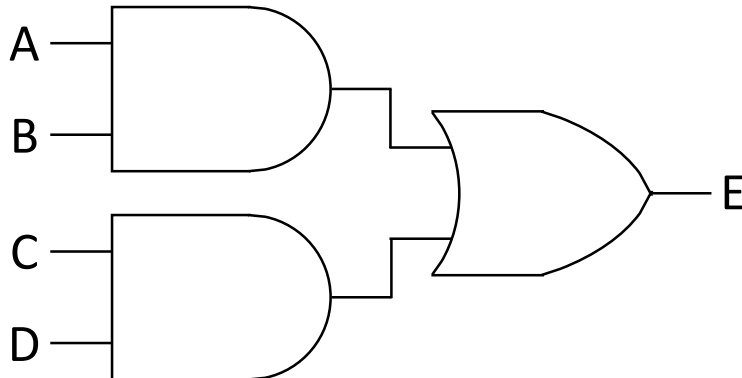
AND



OR

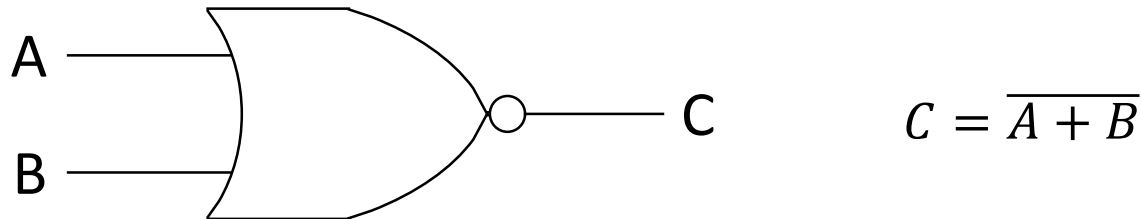


Sum of Products

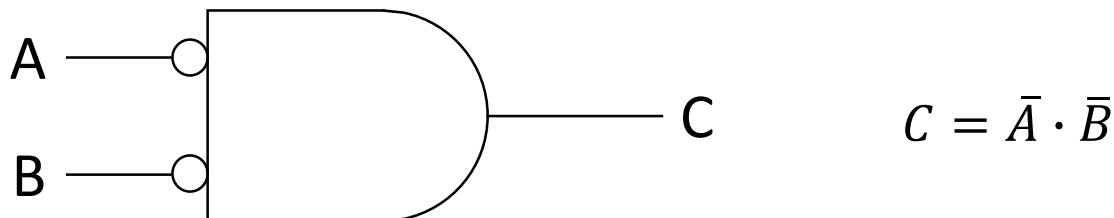


NOR Gates

- A **NOR Gate** is a logic gate that implements an OR operation followed by a NOT operation
 - Implementing NOR with transistors is cheaper than implementing OR and NOT separately then cascading

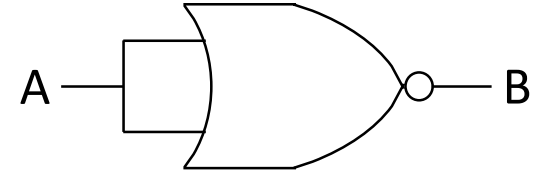
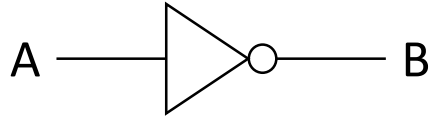


- Applying DeMorgan's laws, a NOR gate is equivalent to an AND gate with inverted inputs

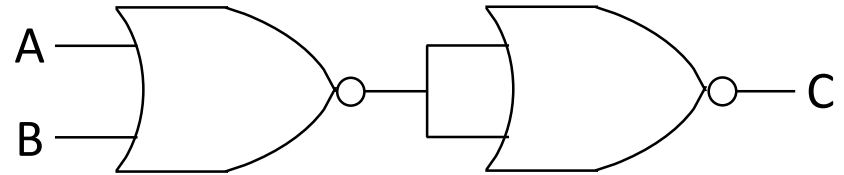
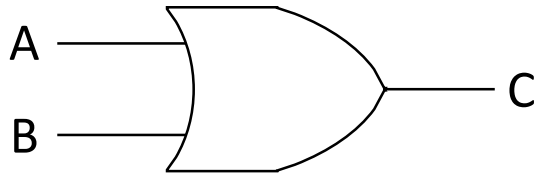


Mapping to NOR Gates

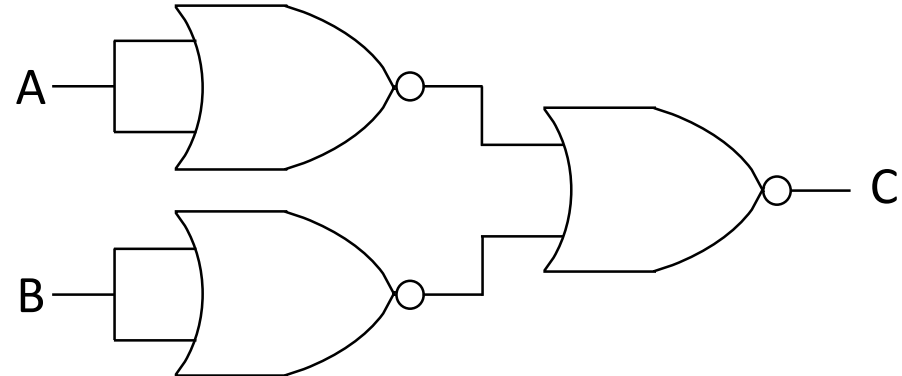
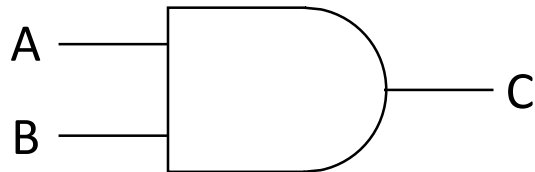
NOT



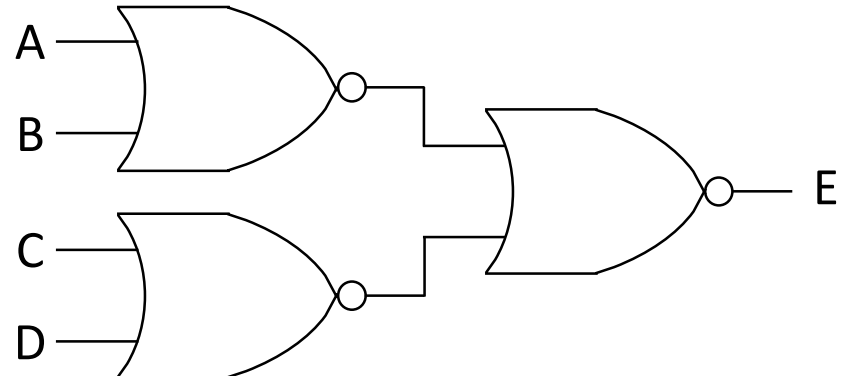
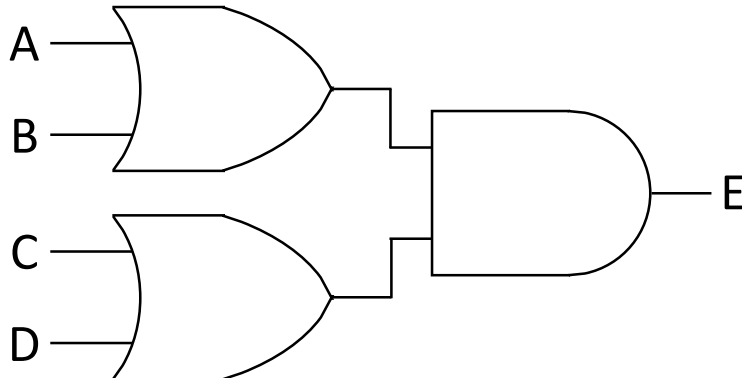
OR



AND



Product
of Sums



Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.2, B.3